



Healthcare Blockchain Project

Amr Saber 221017780

Youssef Motaz 221017608

Mohamed Mostafa 221027785

1.Introduction:

Healthcare data is among the most sensitive categories of information, requiring strict confidentiality, availability, and integrity. Traditional healthcare information systems rely on centralized databases that expose several weaknesses including single points of failure, lack of transparency, high breach impact, and limited verifiability. Blockchain technology introduces a distributed ledger model capable of resolving many of these issues by offering immutability, decentralization, and verifiable audit trails. This report

introduces a blockchain based healthcare records system designed using Ethereum smart contracts and IPFS for decentralized data storage. The system focuses on secure record submission, patient-controlled authorization, role based access management, and tamper-proof metadata logs. The remainder of this report presents related work, system architecture, threat analysis, implementation details, security guarantees, and performance evaluation.

2. Related Work:

Blockchain in healthcare has gained significant attention, with research focusing on secure medical data exchange, audit trails, identity management, and interoperability. Several frameworks have been proposed:

- MedRec: An early blockchain model enabling medical record sharing using metadata stored on-chain.
- Guardtime KSI: A blockchain inspired integrity solution deployed at national scale.

- OmniPHR: A decentralized personal health record framework ensuring patient ownership. However, many proposed solutions suffer from high gas costs, complex interoperability requirements, or lack of fine-grained access control.

Our solution improves on these by using minimal on-chain storage, role-based permissions using OpenZeppelin AccessControl, and lightweight IPFS linkage.

3. System Model & Threat Model:

The system comprises three main entities:

1) Admin: Responsible for registering healthcare providers and patients.

2) Provider: Authorized medical staff who submit medical records.

3) Patient: Data owner with the ability to grant or revoke access. The workflow follows:

- **Providers generate medical records, hash them using Keccak256, and upload full documents to IPFS.**

- **Smart contracts store metadata including record hash, CID, timestamp, and authorized parties.**

- **Patients maintain ultimate control over record access.**

Threat Model: The system

defends against:

- **Unauthorized access (malicious parties reading unpermitted records)**

- **Data tampering (records modified by attackers)**

- **Replay attacks (reusing old access permissions)**

- **Provider impersonation**

Assumptions:

- **Ethereum network is Byzantine fault tolerant.**
- **Users secure their private keys.**
- **IPFS is used only for storage; authenticity is verified through the on chain hash**

4. Smart Contract Architecture:

The smart contract architecture follows a modular RBAC based design:

- AccessControl from OpenZeppelin defines ADMIN_ROLE, PROVIDER_ROLE, and PATIENT_ROLE.
- A Record struct holds: provider, patient, recordHash, ipfsCid,

timestamp.

- A mapping(bytes32 => Record) stores all record metadata.
- A nested mapping accessList[recordId][user] tracks access permissions.
- Events emit RecordSubmitted, AccessGranted, AccessRevoked.

Flowchart (Text Representation):

Patient → Grant Access → Smart Contract →

Update Mapping → Provider Reads Record

Table 1 summarizes key storage structures

RBAC based design:

- AccessControl from OpenZeppelin defines ADMIN_ROLE, PROVIDER_ROLE, and PATIENT_ROLE.
 - A Record struct holds: provider, patient, recordHash, ipfsCid, timestamp.
 - A mapping(bytes32 => Record) stores all record metadata.
 - A nested mapping accessList[recordId][user] tracks access permissions.
 - Events emit RecordSubmitted, AccessGranted, AccessRevoked.
- Flowchart (Text Representation):

Patient → Grant Access

→ Smart Contract →

Update Mapping →

Provider Reads Record

Table 1 summarizes key storage structures

Figure 2. Definition of role hierarchy showing ADMIN, PROVIDER, and PATIENT smart contract roles.

```
contract HealthRecords is AccessControl {  
    bytes32 public constant ADMIN_ROLE = k  
    bytes32 public constant PROVIDER_ROLE =  
    bytes32 public constant PATIENT_ROLE =
```

Figure 3. Data structure used to store medical record metadata on-chain.

```

struct Record {
    bytes32 id;
    address provider;
    address patient;
    bytes32 recordHash; // hash of record
    string ipfsCid;      // optional
    uint256 timestamp;
}

```

Figure 4. Mappings for storing medical records and access permissions.

```

// recordId => Record
mapping(bytes32 => Record) public recordId
// recordId => grantee => bool
mapping(bytes32 => mapping(address => bool)) public grantee

```

5.

Implementation:

The proposed system was implemented as a smart contract written in the Solidity programming language (version

0.8.19). The contract logic, role management, and access-control policies were fully realized at the blockchain layer. For development, deployment, debugging, and execution, the Remix IDE was used due to its suitability for rapid prototyping and its built-in Ethereum Virtual Machine (Remix VM) simulation.

During testing, the contract was

deployed on the Remix VM (Prague) environment, which provides multiple Ethereum wallet accounts with pre-funded balances, enabling the emulation of different system actors (administrator, healthcare provider, and patient). All functionality—including role assignment, record submission, access granting, and permission verification—was

tested directly through Remix's transaction interface.

We validated the implementation through a series of functional tests using the Remix VM (Prague) environment. Three distinct Ethereum accounts were used to emulate the different roles:

- Account A – Administrator

- Account B – Healthcare provider
- Account C – Patient

The test scenario proceeded as follows:

- 1) The administrator deployed the contract and registered Account B as a provider and Account C as a patient.
- 2) Acting as the provider, Account B

computed a hash of a sample medical report, uploaded the document to IPFS, and invoked ``submitRecord(C, recordHash, ipfsCid)``. This created a new ``recordId`` and emitted a ``RecordSubmitted`` event.

- 3) Acting as the patient, Account C called ``grantAccess(recordId, B)`` to explicitly authorize the provider (and optionally other

addresses) to read the record.

4) Acting as the authorized provider, Account B invoked ``getRecordMeta(recordId)`` and successfully retrieved the stored metadata.

5) When an unauthorized account attempted to call ``getRecordMeta`` or when ``hasAccess(recordId, unauthorisedAddress)`` was queried, the contract either

returned ``false`` or reverted due to insufficient permissions.

Screenshots from the Remix console and the ``RecordSubmitted`` / ``AccessGranted`` events confirm that the implementation behaves as expected under normal use and enforces the specified access control policies.

6. Security

Analysis:

This section evaluates the robustness of the proposed blockchain-based healthcare record system against major attacks. The core of the security model relies on

(1) decentralized authentication using Ethereum addresses,

(2) role-based access enforcement using OpenZeppelin AccessControl.

(3) integrity guarantees via on-chain hashing. The system does not depend on centralized credentials or passwords, thereby eliminating several traditional attack vectors.

6.1 Resistance to Impersonation Attack:

An impersonation attack occurs when an adversary attempts to act on behalf of another user. In conventional e-health systems, this can be achieved by credential theft or database tampering. In the proposed system, impersonation is prevented because:

- Each user is uniquely identified by their Ethereum wallet address, cryptographically

tied to their private key.

- No passwords or centralized credentials are stored on-chain; authentication is inherently decentralized.
- Role assignments (e.g., PROVIDER_ROLE, PATIENT_ROLE) can only be granted by the administrator and are validated on-chain.

Therefore, an attacker cannot

impersonate a registered provider or patient unless they possess the victim's private key, which aligns the system's security with the underlying ECDSA cryptography of Ethereum.

6.2 Resistance to Modification Attack:

A modification attack refers to altering stored medical information to mislead diagnosis or medical history.

The proposed system prevents such modification due to:

- Medical records are stored as Keccak256 hashes on-chain.
- Any attempt to alter the off-chain file produces a different hash, which fails verification.
- The original record cannot be overwritten; a new record submission generates a new recordId.

- The blockchain ledger ensures immutability—modifying past entries is computationally infeasible.

Thus, modification attempts are detectable immediately by hash mismatch, and permanent data tampering on the blockchain is practically impossible.

6.3 Resistance to Replay Attack:

Replay attacks occur when a valid transaction is captured and later resent by an attacker to gain unauthorized access. The proposed design mitigates this risk by:

- The `submitRecord`` function logs a unique `recordId` and timestamp; replaying the same transaction fails because the `recordId` must be unique.

- Access permissions are explicitly controlled and stored in ``accessList``. A replayed authorization transaction cannot introduce additional access unless the patient themselves authorises it.
- The ``timestamp`` field allows off-chain services to reject stale or duplicated submissions if needed.
- Blockchain signatures inherently include nonce

values, preventing replay of raw transactions at the protocol layer.

Therefore, replay attempts neither allow unauthorized record submissions nor escalate access privileges.

6.4 Summary of Security Guarantees:

The combination of on-chain hashing,

decentralized identity management, access control enforcement, and immutable logs ensures that the system fulfills the key security objectives:

- Authentication → tied to private-key ownership
- Integrity → assured via Keccak256 hashing and immutability
- Authorization → enforced via

AccessControl and nested mappings

- Non-repudiation → supported by blockchain event logs

These properties collectively offer strong protection against impersonation, modification, and replay attacks.

7. Performance

Evaluation:

7.1 Gas

Consumption:

The system stores only metadata (hash + IPFS CID), not

medical files. This significantly reduces gas cost.

- Deployment: Highest one-time cost.
- submitRecord: Moderate cost due to storing struct + emitting event.
- grantAccess: Low cost (single boolean write).
- getRecordMeta/has Access: Very low cost (read-only operations).

Overall, gas usage is efficient because the contract avoids on-chain file storage.

7.2 Latency

Latency depends on blockchain confirmation time:

- submitRecord: One transaction → confirmed per block.
- grantAccess: One transaction → same latency.
- Read operations (getRecordMeta/has Access): Instant, no mining required.

In public networks, typical delay is 5–15 seconds. In Remix VM, calls are instantaneous.

7.3 Summary:

the system is gas-efficient, and transaction latency is acceptable for real healthcare workflows.

8. Conclusion:

This report presented a complete blockchain based healthcare record management system featuring secure

smart contracts, decentralized storage, patient centric permissions, and event based auditability. The architecture demonstrates strong security guarantees through role based authorization and cryptographic hashing while achieving scalability via IPFS. The performance results show that blockchain is a feasible foundation for medical record integrity and controlled sharing. Future extensions may include emergency access protocols, attribute based encryption, machine learning

assisted anomaly detection, and interoperability with EHR standards.

Medical Data Sharing,” IEEE Blockchain, 2016. [3] OpenZeppelin Documentation, AccessControl Module. [4] IPFS Protocol Documentation.

9. References:

[1] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, “An Overview of Blockchain Technology.” [2] A. Azaria et al., “MedRec: Using Blockchain for